

# **LAPORAN TUGAS 3 (1 - 3)**

## **SISTEM OPERASI**



**Dosen Pengampu:**

**Triawan Adi Cahyanto, M.Kom**

**Disusun Oleh:**

**Yoanre Louis A.H.P (2410651039)**

**PRODI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER**

**2025**

## **BAB 1**

### **PENDAHULUAN**

#### **1.1 LATAR BELAKANG**

Perkembangan teknologi komputasi modern menuntut peningkatan efisiensi dan kecepatan pemrosesan data. Sistem operasi sebagai pengelola sumber daya komputasi memegang peran krusial dalam mengoptimalkan penggunaan prosesor, terutama dengan adanya prosesor multi-core yang memungkinkan eksekusi tugas secara paralel. Praktikum ini berfokus pada implementasi thread dan konkurensi dalam pemrograman C, yang merupakan fondasi bagi pengembangan aplikasi modern yang memanfaatkan kemampuan paralelisme. Melalui eksperimen dengan thread, shared memory, sinkronisasi, dan pola producer-consumer, mahasiswa dapat memahami cara kerja multitasking serta tantangan yang muncul seperti race condition dan deadlock. Pemahaman ini sangat penting untuk pengembangan perangkat lunak yang skalabel dan responsif, seperti server web, sistem basis data, dan aplikasi real-time. Dengan mempelajari mekanisme thread dan sinkronisasi, mahasiswa tidak hanya menguasai konsep teoretis tetapi juga mampu menerapkannya dalam menyelesaikan masalah komputasi yang kompleks.

#### **1.2 RUMUSAN MASALAH**

1. Bagaimana mengimplementasikan kalkulator paralel menggunakan thread untuk menjalankan operasi aritmetika secara bersamaan?
2. Bagaimana membagi tugas pemrosesan file teks menjadi beberapa thread dan membandingkan kinerjanya dengan pendekatan single-thread?
3. Bagaimana mensimulasikan pola producer-consumer dengan sinkronisasi yang aman menggunakan mutex?

#### **1.3 TUJUAN**

1. Membuat program kalkulator paralel yang memanfaatkan multi-threading untuk meningkatkan throughput komputasi.
2. Menganalisis perbandingan kinerja antara pemrosesan file multi-thread dan single-thread.

3. Mengimplementasikan simulasi producer-consumer dengan sinkronisasi untuk menghindari race condition.

## BAB II

### PEMBAHASAN

#### 2.1 PRAKTIK TUGAS 1

Program kalkulator paralel ini merupakan implementasi konkurensi yang memanfaatkan multiple thread untuk menjalankan operasi matematika secara bersamaan. Kode menggunakan struktur data `KalkulatorData` yang berfungsi sebagai wadah untuk menyimpan dua angka input, hasil perhitungan, dan nama operasi. Empat thread terpisah dibuat untuk menangani masing-masing operasi aritmetika: penjumlahan, pengurangan, perkalian, dan pembagian. Setiap thread menerima pointer ke struktur data sendiri melalui parameter `void\* arg` yang kemudian dicast ke tipe yang sesuai. Mekanisme ini memastikan isolasi data antar thread sehingga menghindari race condition tanpa perlu mutex.

Fungsi `pthread\_create()` digunakan untuk inisiasi thread yang menjalankan fungsi operasi terkait, sementara `pthread\_join()` menjamin main thread menunggu semua perhitungan selesai sebelum menampilkan output akhir. Pada operasi pembagian, terdapat penanganan error sederhana untuk kasus pembagian dengan nol. Program ini mendemonstrasikan paralelisme sejati dimana semua komputasi dapat berjalan bersamaan, memanfaatkan potensi prosesor multicore untuk meningkatkan throughput secara signifikan dibanding pendekatan sekuensial.

##### 2.1.1 Membuat Kalkulator Paralel

```
dany@LAPTOP-GRJL2V1D:~$ nano kalkulator_paralel.c
dany@LAPTOP-GRJL2V1D:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -pthread
```

```
dany@LAPTOP-GRJL2V1D:~$ ./kalkulator_paralel
=== KALKULATOR PARALEL ===
Masukkan angka pertama: 10
Masukkan angka kedua: 5

Memulai perhitungan paralel...

Thread Penjumlahan: 10.00 + 5.00 = 15.00
Thread Pengurangan: 10.00 - 5.00 = 5.00
Thread Perkalian: 10.00 * 5.00 = 50.00
Thread Pembagian: 10.00 / 5.00 = 2.00

=== HASIL AKHIR ===
Penjumlahan: 15.00
Pengurangan: 5.00
Perkalian: 50.00
Pembagian: 2.00
dany@LAPTOP-GRJL2V1D:~$ |
```

### 2.1.2 Penejelasan Proses Kompilasi dan Eksekusi

#### A. PROSES KOMPILASI:

- File kalkulator\_parallel.c pertama kali diproses untuk memasukkan semua library yang diperlukan seperti stdio.h dan pthread.h
- Kemudian kode C diubah menjadi bahasa assembly agar lebih dekat dengan bahasa mesin
- Dari assembly, kode dikonversi lagi menjadi binary code atau kode mesin yang benar-benar bisa dieksekusi
- Terakhir, semua bagian program disatukan dengan library pthread yang khusus untuk menangani thread
- Hasil akhirnya adalah file executable kalkulator\_parallel yang siap dijalankan

#### B. AWAL PROGRAM BERJALAN:

- Saat program dijalankan, sistem operasi memuatnya ke memory dan memulai proses dari fungsi main()
- Program akan meminta user memasukkan dua angka yang ingin dihitung
- Kedua angka ini disimpan dalam variabel sementara untuk persiapan perhitungan
- Program menyiapkan empat wadah data terpisah untuk masing-masing operasi matematika

#### C. PEMBUATAN DAN JALANNYA THREAD:

- Program membuat empat thread sekaligus yang bekerja paralel seperti punya empat pekerja yang mengerjakan tugas berbeda
- Thread pertama khusus menangani penjumlahan, thread kedua pengurangan, thread ketiga perkalian, dan thread keempat pembagian
- Setiap thread langsung bekerja menghitung hasil operasinya masing-masing menggunakan angka yang sama
- Untuk pembagian, ada pengecekan khusus kalau angka kedua adalah nol agar tidak terjadi error

#### D. PENYELARASAN HASIL:

- Program utama menunggu sampai keempat thread selesai mengerjakan tugasnya
- Setelah semua thread selesai, hasil dari masingmasing operasi dikumpulkan
- Karena setiap thread punya data sendirisendiri, tidak ada tabrakan data antar thread

#### E. TAMPILAN HASIL DAN PENUTUP:

- Semua hasil perhitungan ditampilkan dalam format yang rapi dengan dua angka di belakang koma
- Semua resource dan memory yang dipakai dibersihkan secara otomatis
- Program selesai dengan status sukses tanpa error

#### F. KODE C:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <stdlib.h>
```

```
// Struktur untuk menyimpan data yang dibagikan ke thread
```

```
typedef struct {
```

```
    double angka1;
```

```
    double angka2;
```

```
    double hasil;
```

```
    char* operasi;
```

```
} KalkulatorData;
```

```
// Fungsi untuk penjumlahan
```

```
void* tambah(void* arg) {
```

```

KalkulatorData* data = (KalkulatorData*) arg;

data>hasil = data>angka1 + data>angka2;

data>operasi = "Penjumlahan";

printf("Thread Penjumlahan: %.2lf + %.2lf = %.2lf\n", data>angka1, data>angka2,
data>hasil);

return NULL;

}

```

// Fungsi untuk pengurangan

```

void* kurang(void* arg) {

    KalkulatorData* data = (KalkulatorData*) arg;

    data>hasil = data>angka1 - data>angka2;

    data>operasi = "Pengurangan";

    printf("Thread Pengurangan: %.2lf - %.2lf = %.2lf\n", data>angka1, data>angka2,
data>hasil);

    return NULL;

}

```

// Fungsi untuk perkalian

```

void* kali(void* arg) {

    KalkulatorData* data = (KalkulatorData*) arg;

    data>hasil = data>angka1 * data>angka2;

    data>operasi = "Perkalian";

    printf("Thread Perkalian: %.2lf * %.2lf = %.2lf\n", data>angka1, data>angka2,
data>hasil);

}

```

```

        return NULL;
    }

// Fungsi untuk pembagian
void* bagi(void* arg) {
    KalkulatorData* data = (KalkulatorData*) arg;

    if (data>angka2 != 0) {
        data>hasil = data>angka1 / data>angka2;

        data>operasi = "Pembagian";

        printf("Thread Pembagian: %.2lf / %.2lf = %.2lf\n", data>angka1, data>angka2,
data>hasil);
    } else {
        data>hasil = 0;

        data>operasi = "Pembagian";

        printf("Thread Pembagian: Error! Pembagian dengan nol.\n");
    }

    return NULL;
}

int main() {
    double angka1, angka2;

    // Meminta input dari user

    printf("=== KALKULATOR PARALEL ===\n");

    printf("Masukkan angka pertama: ");

```



```
scanf("%lf", &angka1);

printf("Masukkan angka kedua: ");

scanf("%lf", &angka2);

printf("\n");


// Deklarasi thread dan data

pthread_t thread_tambah, thread_kurang, thread_kali, thread_bagi;

KalkulatorData data_tambah, data_kurang, data_kali, data_bagi;


// Inisialisasi data untuk setiap thread

data_tambah.angka1 = data_kurang.angka1 = data_kali.angka1 = data_bagi.angka1 =
angka1;

data_tambah.angka2 = data_kurang.angka2 = data_kali.angka2 = data_bagi.angka2 =
angka2;


printf("Memulai perhitungan paralel...\n\n");


// Membuat thread untuk setiap operasi

pthread_create(&thread_tambah, NULL, tambah, &data_tambah);

pthread_create(&thread_kurang, NULL, kurang, &data_kurang);

pthread_create(&thread_kali, NULL, kali, &data_kali);

pthread_create(&thread_bagi, NULL, bagi, &data_bagi);


// Menunggu semua thread selesai

pthread_join(thread_tambah, NULL);
```

```

pthread_join(thread_kurang, NULL);

pthread_join(thread_kali, NULL);

pthread_join(thread_bagi, NULL);


// Menampilkan hasil akhir

printf("\n=== HASIL AKHIR ===\n");

printf("%s: %.2lf\n", data_tambah.operasi, data_tambah.hasil);

printf("%s: %.2lf\n", data_kurang.operasi, data_kurang.hasil);

printf("%s: %.2lf\n", data_kali.operasi, data_kali.hasil);

printf("%s: %.2lf\n", data_bagi.operasi, data_bagi.hasil);


return 0;

}

```

## 2.2 PRAKTIK TUGAS 2

Program file processing paralel ini mengimplementasikan pendekatan multithreading untuk menghitung jumlah kata dalam file teks secara efisien. Kode menggunakan struktur `ThreadData` untuk membagi workload dimana setiap thread bertanggung jawab memproses separuh bagian teks. Algoritma penghitungan kata bekerja dengan melacak status `in\_word` yang mendeteksi transisi antara karakter spasi dan nonspasi. Pembagian teks dioptimalkan dengan memastikan pemisahan tidak memotong kata di tengah dengan mencari batas spasi atau newline terdekat.

Dua thread dibuat menggunakan `pthread\_create()` untuk menjalankan fungsi `hitung\_kata\_thread()` secara paralel, masing-masing mengoperasikan pada bagian teks yang berbeda. Program mengukur waktu eksekusi menggunakan `clock()` untuk kedua versi multithread dan singlethread, memungkinkan perbandingan performa yang akurat. Setelah kedua thread selesai dijoin, hasil parsial digabungkan dan dibandingkan dengan hasil

singlethread yang memproses seluruh teks secara sekuensial. Implementasi ini mendemonstrasikan bagaimana paralelisme dapat meningkatkan throughput pemrosesan data dengan membagi tugas besar menjadi subunit yang dapat dieksekusi bersamaan, meskipun overhead sinkronisasi mungkin mempengaruhi efisiensi pada teks berukuran kecil.

### 2.2.1 Membuat File Processing

- Otomatis

```
dany@LAPTOP-GRJL2V1D:~$ nano file_processing.c
dany@LAPTOP-GRJL2V1D:~$ gcc file_processing.c -o file_processing -pthread
```

```
dany@LAPTOP-GRJL2V1D:~$ ./file_processing
File contoh.txt berhasil dibuat
=== FILE PROCESSING PARALEL ===
Panjang teks: 312 karakter

--- VERSI MULTI-THREAD ---
Thread: Menghitung kata dari indeks 0-157 = 24 kata
Thread: Menghitung kata dari indeks 157-312 = 19 kata
Total kata (multi-thread): 43
Waktu eksekusi: 0.000218 detik

--- VERSI SINGLE-THREAD ---
Total kata (single-thread): 43
Waktu eksekusi: 0.000003 detik

=== PERBANDINGAN ===
Multi-thread: 43 kata, 0.000218 detik
Single-thread: 43 kata, 0.000003 detik
KESIMPULAN: Single-thread lebih cepat 0.000215 detik
dany@LAPTOP-GRJL2V1D:~$
```

- Manual Menggunakan input.txt

```
dany@LAPTOP-GRJL2V1D:~$ nano input.txt
dany@LAPTOP-GRJL2V1D:~$ nano file_processing.c
dany@LAPTOP-GRJL2V1D:~$ gcc file_processing.c -o file_processing -pthread
dany@LAPTOP-GRJL2V1D:~$ ./file_processing
File contoh.txt berhasil dibuat
=== FILE PROCESSING PARALEL ===
Panjang teks: 202 karakter

--- VERSI MULTI-THREAD ---
Thread: Menghitung kata dari indeks 0-104 = 18 kata
Thread: Menghitung kata dari indeks 104-202 = 17 kata
Total kata (multi-thread): 35
Waktu eksekusi: 0.000357 detik

--- VERSI SINGLE-THREAD ---
Total kata (single-thread): 35
Waktu eksekusi: 0.000018 detik

=== PERBANDINGAN ===
Multi-thread: 35 kata, 0.000357 detik
Single-thread: 35 kata, 0.000018 detik
KESIMPULAN: Single-thread lebih cepat 0.000339 detik
dany@LAPTOP-GRJL2V1D:~$
```

### 2.2.2 Penjelasan Proses Kompilasi dan Eksekusi

#### A. PROSES KOMPILASI:

- Pertama, file source code kita yang berekstensi .c akan melalui tahap preprocessing dimana semua header files seperti stdio.h dan pthread.h akan diekspansi
- Setelah itu, kode C yang sudah bersih akan dikompilasi menjadi bahasa assembly oleh GCC compiler
- Kode assembly kemudian diubah menjadi binary code atau object code yang bisa dimengerti mesin

- Terakhir, semua object code dilink dengan library yang diperlukan, khususnya pthread library untuk fitur threading
- Hasil akhirnya adalah file executable yang siap dijalankan di sistem operasi

#### B. PROSES EKSEKUSI AWAL:

- Ketika program dijalankan, sistem operasi akan memuatnya ke memory dan memulai eksekusi dari fungsi main()
- Program pertama kali akan membuat file contoh otomatis bernama "contoh.txt" yang berisi teks latihan
- Kemudian program membaca seluruh isi file tersebut dan menyimpannya di memory
- Program juga menghitung panjang teks total untuk persiapan pembagian kerja

#### C. PROSES MULTITHREAD:

- Program membagi teks menjadi dua bagian sama besar, tapi dengan trik khusus agar tidak memotong kata di tengah-tengah
- Dua thread dibuat untuk bekerja paralel: thread pertama menghitung kata separuh bagian awal, thread kedua menghitung separuh bagian akhir
- Setiap thread bekerja dengan cara memindai setiap karakter dan mendeteksi kapan sebuah kata dimulai dan berakhir
- Waktu eksekusi dicatat untuk melihat seberapa cepat kedua thread menyelesaikan tugasnya

#### D. PERBANDINGAN SINGLETHREAD:

- Untuk perbandingan, program juga menjalankan versi singlethread yang menghitung semua kata secara berurutan
- Waktu eksekusi versi singlethread ini juga diukur dengan teliti
- Hasil kedua metode dicek apakah sama untuk memastikan perhitungannya akurat

#### E. ANALISIS HASIL:

- Hasil dari kedua thread dijumlahkan untuk mendapatkan total kata versi multithread

- Program membandingkan waktu eksekusi kedua metode dan menentukan mana yang lebih cepat
- Kesimpulan ditampilkan tentang efisiensi penggunaan multithreading
- Terakhir, memory yang dipakai dibersihkan dan program selesai dijalankan

#### F. KODE C:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

```
#include <sys/stat.h>
```

```
// Struktur untuk data thread
```

```
typedef struct {
```

```
    char* text;
```

```
    int start;
```

```
    int end;
```

```
    int word_count;
```

```
} ThreadData;
```

```
// Fungsi menghitung kata dalam bagian teks
```

```
int hitung_kata_bagian(char* text, int start, int end) {
```

```
    int count = 0;
```

```
    int in_word = 0;
```

```

for (int i = start; i < end; i++) {

    if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t' || text[i] == '\0') {

        if (in_word) {

            count++;

            in_word = 0;

        }

    } else {

        in_word = 1;

    }

}

// Hitung kata terakhir jika teks berakhir tanpa spasi

if (in_word) {

    count++;

}

return count;

}

// Fungsi thread untuk menghitung kata

void* hitung_kata_thread(void* arg) {

    ThreadData* data = (ThreadData*) arg;

    data->word_count = hitung_kata_bagian(data->text, data->start, data->end);

```

```

printf("Thread: Menghitung kata dari indeks %d%d = %d kata\n",
      data>start, data>end, data>word_count);

return NULL;
}

```

```

// Fungsi singlethread (versi biasa)

```

```

int hitung_kata_singlethread(char* text) {
    return hitung_kata_bagian(text, 0, strlen(text));
}

```

```

// Fungsi baca file

```

```

char* baca_file(const char* filename) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("Error: Tidak bisa membuka file %s\n", filename);
        return NULL;
    }
}

```

```

// Cari ukuran file

```

```

fseek(file, 0, SEEK_END);

long size = ftell(file);

fseek(file, 0, SEEK_SET);

```

```

// Allokasi memori untuk teks

```

```

char* text = malloc(size + 1);

fread(text, 1, size, file);

text[size] = '\0';


fclose(file);

return text;
}


// Fungsi buat file contoh

void buat_file_contoh() {

    FILE* file = fopen("contoh.txt", "w");

    if (file) {

        fprintf(file, "Ini adalah file teks contoh untuk praktikum sistem operasi\n");

        fprintf(file, "Kita akan menghitung jumlah kata dalam file ini\n");

        fprintf(file, "Menggunakan dua thread yang bekerja paralel\n");

        fprintf(file, "Thread pertama menghitung separuh bagian awal\n");

        fprintf(file, "Thread kedua menghitung separuh bagian akhir\n");

        fprintf(file, "Hasilnya akan digabungkan dan dibandingkan dengan versi singlethread\n");

        fclose(file);

        printf("File contoh.txt berhasil dibuat\n");

    }

}


int main() {

```



```

// Buat file contoh otomatis

buat_file_contoh();


// Baca file

char* text = baca_file("contoh.txt");

if (!text) {

    return 1;

}


int text_length = strlen(text);

printf("=== FILE PROCESSING PARALEL ===\n");

printf("Panjang teks: %d karakter\n\n", text_length);


// Versi MULTITHREAD

printf(" VERSI MULTITHREAD \n");

clock_t start_multi = clock();


pthread_t thread1, thread2;

ThreadData data1, data2;


// Bagi teks menjadi 2 bagian

int mid = text_length / 2;


// Pastikan pembagian tidak memotong kata di tengah

```

```
while (mid < text_length && text[mid] != ' ' && text[mid] != '\n') {  
    mid++;  
}  
  
data1.text = text;  
data1.start = 0;  
data1.end = mid;  
  
data2.text = text;  
data2.start = mid;  
data2.end = text_length;  
  
// Buat thread  
  
pthread_create(&thread1, NULL, hitung_kata_thread, &data1);  
pthread_create(&thread2, NULL, hitung_kata_thread, &data2);  
  
// Tunggu thread selesai  
  
pthread_join(thread1, NULL);  
pthread_join(thread2, NULL);  
  
// Gabungkan hasil  
  
int total_multi = data1.word_count + data2.word_count;  
  
clock_t end_multi = clock();  
  
double waktu_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;
```

```
printf("Total kata (multithread): %d\n", total_multi);
```

```
printf("Waktu eksekusi: %.6f detik\n\n", waktu_multi);
```

```
// Versi SINGLETHREAD
```

```
printf(" VERSI SINGLETHREAD \n");
```

```
clock_t start_single = clock();
```

```
int total_single = hitung_kata_singlethread(text);
```

```
clock_t end_single = clock();
```

```
double waktu_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;
```

```
printf("Total kata (singlethread): %d\n", total_single);
```

```
printf("Waktu eksekusi: %.6f detik\n\n", waktu_single);
```

```
// Perbandingan
```

```
printf("=== PERBANDINGAN ===\n");
```

```
printf("Multithread: %d kata, %.6f detik\n", total_multi, waktu_multi);
```

```
printf("Singlethread: %d kata, %.6f detik\n", total_single, waktu_single);
```

```
if (waktu_multi < waktu_single) {
```

```
    printf("KESIMPULAN: Multithread lebih cepat %.6f detik\n",
```

```
        waktu_single - waktu_multi);
```

```
} else {
```

```

printf("KESIMPULAN: Singlethread lebih cepat %.6f detik\n",
      waktu_multi waktu_single);

}

// Bersihkan memori

free(text);

return 0;

}

```

### 2.3 PRAKTIK TUGAS 3

Program producerconsumer ini mensimulasikan dua thread yang bekerja sama dengan buffer terbatas berkapasitas 10 item. Producer bertugas menghasilkan angka acak antara 1100 dan memasukkannya ke buffer, sementara consumer mengambil angka tersebut untuk diproses dengan menghitung kuadratnya. Kedua thread berkomunikasi melalui buffer bersama yang dilindungi mutex untuk mencegah race condition. Producer akan menunggu jika buffer penuh, sedangkan consumer menunggu ketika buffer kosong, menciptakan mekanisme flow control yang natural.

Program menggunakan circular buffer dengan variabel in dan out yang berputar menggunakan modulo operation untuk efisiensi memory. Producer bekerja lebih cepat dengan delay 100ms, sementara consumer sengaja diperlambat 150ms untuk mensimulasikan perbedaan kecepatan pemrosesan. Setelah menghasilkan 20 item, kedua thread berhenti secara sinkron. Simulasi ini secara efektif mendemonstrasikan konsep synchronisasi, mutual exclusion, dan koordinasi antar thread dalam sistem konkuren.

### 2.3.1 Membuat Procedur Consumer

```
dany@LAPTOP-GRJL2V1D:~$ nano producer_consumer.c
dany@LAPTOP-GRJL2V1D:~$ gcc producer_consumer.c -o producer_consumer -pthread
```

```
dany@LAPTOP-GRJL2V1D:~$ ./producer_consumer
=== PRODUCER-CONSUMER SEDERHANA ===
PRODUCER: Item 1 | Buffer: 1/10
CONSUMER: Proses 1 | Buffer: 0/10
PRODUCER: Item 2 | Buffer: 1/10
PRODUCER: Item 3 | Buffer: 2/10
CONSUMER: Proses 3 | Buffer: 1/10
PRODUCER: Item 4 | Buffer: 2/10
PRODUCER: Item 5 | Buffer: 3/10
CONSUMER: Proses 5 | Buffer: 2/10
PRODUCER: Item 6 | Buffer: 3/10
CONSUMER: Proses 6 | Buffer: 2/10
PRODUCER: Item 7 | Buffer: 3/10
PRODUCER: Item 8 | Buffer: 4/10
CONSUMER: Proses 8 | Buffer: 3/10
PRODUCER: Item 9 | Buffer: 4/10
PRODUCER: Item 10 | Buffer: 5/10
CONSUMER: Proses 10 | Buffer: 4/10
PRODUCER: Item 11 | Buffer: 5/10
PRODUCER: Item 12 | Buffer: 6/10
CONSUMER: Proses 12 | Buffer: 5/10
PRODUCER: Item 13 | Buffer: 6/10
PRODUCER: Item 14 | Buffer: 7/10
CONSUMER: Proses 14 | Buffer: 6/10
PRODUCER: Item 15 | Buffer: 7/10
CONSUMER: Proses 15 | Buffer: 6/10
CONSUMER: Proses 13 | Buffer: 5/10
CONSUMER: Proses 11 | Buffer: 4/10
CONSUMER: Proses 9 | Buffer: 3/10
CONSUMER: Proses 7 | Buffer: 2/10
CONSUMER: Proses 4 | Buffer: 1/10
CONSUMER: Proses 2 | Buffer: 0/10
dany@LAPTOP-GRJL2V1D:~$ |
```

### 2.3.2 Penjelasan Proses Kompilasi dan Eksekusi

#### A. PROSES KOMPILASI:

- File producer\_consumer.c diproses untuk memasukkan semua library yang diperlukan seperti pthread.h dan unistd.h
- Kode C diubah menjadi bahasa assembly agar lebih mudah dipahami komputer
- Kode assembly kemudian dikonversi menjadi binary code atau kode mesin
- Semua bagian program disatukan dengan library pthread untuk fitur thread dan library standar C
- Hasil akhirnya file executable producer\_consumer yang siap dijalankan

#### B. PERSIAPAN PROGRAM:

- Saat program dijalankan, sistem memuatnya ke memory dan memulai dari fungsi main()
- Program mengatur generator angka random agar menghasilkan angka yang berbeda setiap kali dijalankan
- Buffer bersama diinisialisasi dengan nilai kosong dan semua slot diisi angka nol
- Program menampilkan informasi tentang ukuran buffer dan total item yang akan diproduksi

### C. PROSES PRODUCER:

- Thread producer mulai bekerja dengan menghasilkan angka random antara 1100
- Sebelum memasukkan ke buffer, producer mengecek dulu apakah buffer masih ada tempat kosong
- Kalau buffer penuh, producer akan menunggu sambil memberi pesan "Buffer penuh!"
- Producer menggunakan mutex seperti kunci agar tidak tabrakan dengan consumer saat mengakses buffer
- Setiap selesai memproduksi item, producer tidur sebentar 100ms

### D. PROSES CONSUMER:

- Thread consumer bekerja mengambil item dari buffer untuk diproses
- Consumer mengecek dulu apakah buffer berisi item yang bisa diambil
- Kalau buffer kosong, consumer menunggu sambil memberi pesan "Buffer kosong!"
- Setiap item yang diambil akan dihitung kuadratnya sebagai simulasi pemrosesan data
- Consumer lebih lambat dari producer dengan tidur 150ms agar terlihat jelas prosesnya

### E. SINKRONISASI DAN AKHIR PROGRAM:

- Program utama menunggu kedua thread selesai bekerja
- Setelah producer menghasilkan 20 item dan consumer memproses 20 item, program berakhir
- Status buffer akhir ditampilkan untuk melihat kondisi terakhir
- Semua resource thread dan memory dibersihkan secara otomatis

### F. KODE C:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
#define BUFFER_SIZE 10
```

```
#define MAX_ITEM 20
```

```
// Struktur buffer bersama
```

```
typedef struct {
```

```
    int buffer[BUFFER_SIZE];
```

```
    int count;
```

```
    int in;
```

```
    int out;
```

```
} Buffer;
```

```
Buffer shared_buffer;
```

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
// Fungsi producer
```

```
void* producer(void* arg) {
```

```
    int produced = 0;
```

```
    while (produced < MAX_ITEM) {
```

```
        // Generate angka random
```

```
        int item = rand() % 100 + 1;
```

```
        pthread_mutex_lock(&mutex);
```

```

// Cek jika buffer penuh

if (shared_buffer.count < BUFFER_SIZE) {

    shared_buffer.buffer[shared_buffer.in] = item;

    shared_buffer.in = (shared_buffer.in + 1) % BUFFER_SIZE;

    shared_buffer.count++;

    printf("PRODUCER: Hasilkan item %d | Buffer: %d/10 item\n",
        item, shared_buffer.count);

    produced++;
} else {

    printf("PRODUCER: Buffer penuh! Menunggu...\n");

}

pthread_mutex_unlock(&mutex);

usleep(100000); // Tidur 100ms

}

printf("PRODUCER: Selesai memproduksi %d item\n", MAX_ITEM);

return NULL;

}

// Fungsi consumer

void* consumer(void* arg) {

```



```
int consumed = 0;

while (consumed < MAX_ITEM) {

    pthread_mutex_lock(&mutex);

    // Cek jika buffer kosong

    if (shared_buffer.count > 0) {

        int item = shared_buffer.buffer[shared_buffer.out];

        shared_buffer.out = (shared_buffer.out + 1) % BUFFER_SIZE;

        shared_buffer.count--;

        // Proses item (kuadratkan)

        int processed = item * item;

        printf("CONSUMER: Ambil item %d -> Proses: %d2 = %d | Buffer: %d/10 item\n",
               item, item, processed, shared_buffer.count);

        consumed++;

    } else {

        printf("CONSUMER: Buffer kosong! Menunggu...\n");

    }

    pthread_mutex_unlock(&mutex);

    usleep(150000); // Tidur 150ms (lebih lambat dari producer)

}
```

```
printf("CONSUMER: Selesai memproses %d item\n", MAX_ITEM);

return NULL;

}
```

// Inisialisasi buffer

```
void init_buffer() {

    shared_buffer.count = 0;

    shared_buffer.in = 0;

    shared_buffer.out = 0;

    for (int i = 0; i < BUFFER_SIZE; i++) {

        shared_buffer.buffer[i] = 0;

    }

}
```

// Fungsi tampilkan status buffer

```
void tampilkan_buffer() {

    printf("\n=== STATUS BUFFER ===\n");

    printf("Count: %d/10\n", shared_buffer.count);

    printf("Isi buffer: [");

    for (int i = 0; i < BUFFER_SIZE; i++) {

        printf("%d", shared_buffer.buffer[i]);

        if (i < BUFFER_SIZE - 1) printf(", ");

    }

}
```

```
printf("]\n");

printf("In: %d, Out: %d\n\n", shared_buffer.in, shared_buffer.out);
}

int main() {

    pthread_t prod_thread, cons_thread;

    // Seed random number generator
    srand(time(NULL));

    // Inisialisasi buffer
    init_buffer();

    printf("=== SIMULASI PRODUCER-CONSUMER ===\n");

    printf("Buffer size: %d item\n", BUFFER_SIZE);

    printf("Total item yang akan diproduksi: %d\n\n", MAX_ITEM);

    // Tampilkan buffer awal
    tampilkan_buffer();

    // Buat thread producer dan consumer
    pthread_create(&prod_thread, NULL, producer, NULL);

    pthread_create(&cons_thread, NULL, consumer, NULL);
```

```
// Tunggu kedua thread selesai

pthread_join(prod_thread, NULL);

pthread_join(cons_thread, NULL);


// Tampilkan buffer akhir

tampilkan_buffer();


printf("=== SIMULASI SELESAI ===\n");


return 0;

}
```

## **BAB III**

### **PENUTUP**

#### **3.1 KESIMPULAN**

Praktikum ini berhasil mengimplementasikan tiga program yang mendemonstrasikan konsep thread dan konkurensi dalam sistem operasi. Program kalkulator paralel membuktikan bahwa pembagian tugas ke dalam thread terpisah dapat meningkatkan efisiensi komputasi. Pemrosesan file dengan multi-thread menunjukkan bahwa paralelisme efektif untuk tugas I/O-bound, meskipun overhead sinkronisasi perlu dipertimbangkan. Simulasi producer-consumer berhasil menerapkan mekanisme sinkronisasi menggunakan mutex untuk mengamankan akses ke resource bersama. Secara keseluruhan, praktikum ini menegaskan pentingnya memahami konkurensi dalam pengembangan perangkat lunak modern, terutama dalam memanfaatkan arsitektur multi-core dan menghindari masalah seperti race condition serta deadlock.